Generative AI Interview Series

Day – 4

1. Explain the RAG pipeline end-to-end?

🔥 Why Do We Need RAG?

Before RAG, systems relied only on models like GPT-4.

❌ Problems with Pure LLMs:

1. 🧱 Outdated Knowledge

- Model trained on past data (e.g., till 2024)
- Cannot know:
  - Latest stock prices
  - New company policies

👉 Example:
Ask: *"Latest commodity price of aluminum?"*
LLM → **May give outdated answer**

---

2. 🎯 Hallucination Problem

- LLMs **guess** when unsure
- They generate **confident but wrong answers**

👉 Example:

- Asking internal company data → LLM may **fabricate**

---

3. 🔒 No Access to Private Data

- LLMs **don't know your internal docs**

- Cannot access:
  - PDFs
  - Company DB
  - Emails

---

4. 💰 Expensive Fine-Tuning

- Updating model = costly + time-consuming
- Not scalable for dynamic data

☑ Solution = RAG (Retrieval-Augmented Generation)

👉 RAG connects LLM with **external knowledge sources**

Key Idea:

"Don’t store knowledge in the model → fetch it when needed"

🧠 Intuition (Simple Example)

Without RAG:

- LLM answers from memory → ❌ Risky

With RAG:

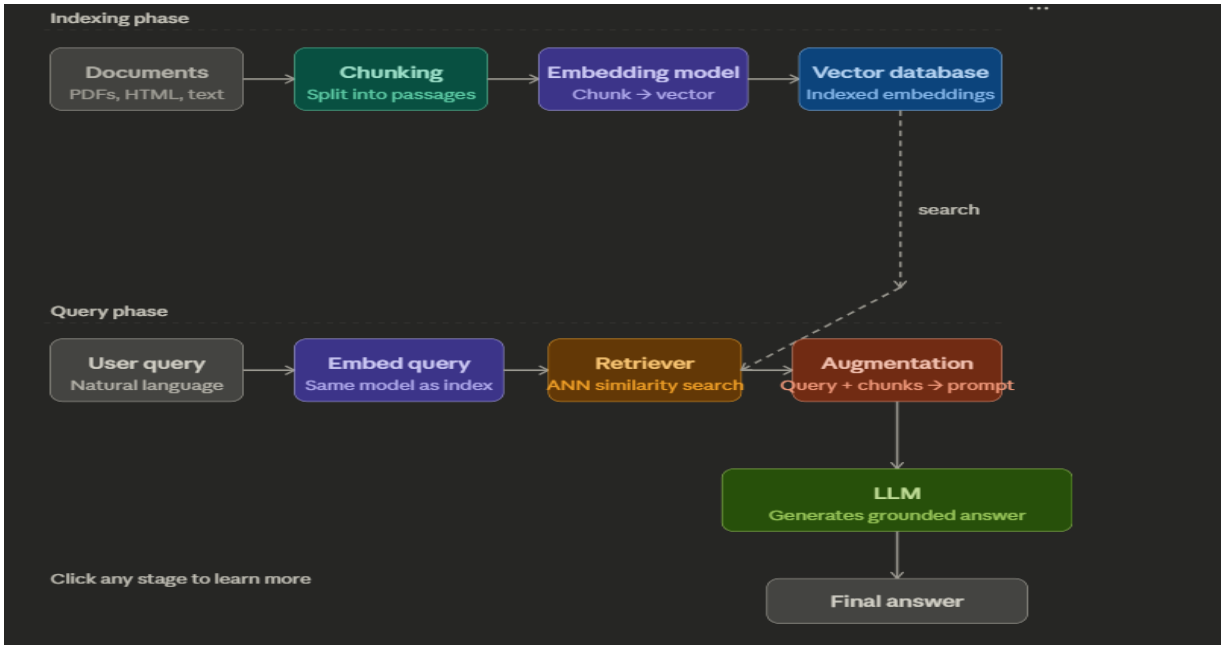
1. Search relevant documents
2. Pass them to LLM
3. Generate grounded answer

👉 Like:

Open-book exam instead of closed-book exam

🔁 RAG Lifecycle (End-to-End)

Now let's go **deep into each stage** 👇



🟣 Step 1: Data Ingestion

📥 Input Sources:

- PDFs
- Databases
- APIs
- Websites

👉 Example (your domain):

- Commodity reports (aluminum, coal, gasoline)

What happens:

- Load documents into system

🟣 Step 2: Text Preprocessing

Clean the data:

- Remove HTML tags
- Lowercase
- Remove noise

Why important?

Better embeddings = better retrieval

What is Chunking?

Split large documents into smaller pieces

Why needed?

- LLM has token limits
- Embeddings work best on smaller text

Types:

1. Fixed chunking (e.g., 500 tokens)
2. Overlapping chunks (best)

Example:

Document:

```
Aluminum prices increased due to supply shortage...
```

Chunks:

- Chunk 1 → "Aluminum prices increased..."
- Chunk 2 → "…due to supply shortage..."

🟢 Step 4: Embedding Generation

👉 Convert text → vectors

Tools:

- OpenAI Embeddings API
- Sentence Transformers

What is embedding?

👉 Text → numerical vector

Example:

```
"Aluminum price rising" → [0.21, -0.45, 0.88, ...]
```

Why?

To measure similarity mathematically

🟤 Step 5: Vector Database Storage

Store embeddings in DB:

Popular DBs:

- Pinecone
- FAISS
- Weaviate

Stored Data:

- Vector
- Original text
- Metadata (source, date, etc.)

🔘 Step 6: Query Input (User Question)

👉 User asks:

```
"What is the reason for aluminum price increase?"
```

🔘 Step 7: Query Embedding

👉 Convert query → vector

```
Query → [0.18, -0.40, 0.91, ...]
```

🔘 Step 8: Similarity Search (Retriever)

Core Idea:

Find **closest vectors**

Techniques:

- Cosine similarity
- Euclidean distance

Output:

Top-K relevant chunks

👉 Example:

- Chunk 1 → supply shortage
- Chunk 2 → energy cost increase

🔘 Step 9: Context Augmentation

👉 Combine:

- User query
- Retrieved chunks

Prompt looks like:

```
Context:
[chunk1]
[chunk2]

Question:
What is aluminum price trend?

Answer:
```

🟣 Step 10: LLM Generation

👉 Use model like GPT-4

What happens:

- Reads context
- Generates grounded answer

Output:

☑ Accurate
☑ Context-aware
☑ Less hallucination

🟣 Step 11: Post-processing (Optional)

- Formatting
- Citations
- Filtering

🔁 Full Flow Summary

```
Data → Chunk → Embedding → Vector DB
                              ↓
User Query → Embedding → Retrieval
                              ↓
                    Context + Query
                              ↓
                            LLM
                              ↓
                         Final Answer
```

🚀 Real-World Example (Your Domain)

Use Case: Commodity Insights (your project)

User asks:
👉 "Why coal prices increased in Q3?"

RAG does:

1. Retrieves internal reports
2. Extracts relevant chunks
3. Sends to LLM
4. Generates grounded answer

💧 Key Advantages of RAG

- ☑ Real-time knowledge
- ☑ No retraining needed
- ☑ Reduced hallucination
- ☑ Works with private data

🧠 Interview Tip (VERY IMPORTANT)

If asked:

👉 "Why RAG over fine-tuning?"

Answer:

Fine-tuning updates model knowledge permanently but is expensive and static. RAG dynamically retrieves latest data, making it scalable, cost-effective, and real-time.

2. What is Chunking in RAG? Different chunking strategies?

◇ 1. What is Chunking?

📌 Definition:

Chunking is the process of breaking large documents into smaller meaningful pieces before converting them into embeddings.

🎯 Why do we need it?

👉 Because:

- LLM has **token limits**
- Embeddings work better on **small text**
- Retrieval becomes **more accurate**

❗ Without Chunking:

```
Full Document → 1 embedding → ❌ poor retrieval
```

☑ With Chunking:

```
Document → Multiple chunks → Better search → Better answer
```

◇ 2. Base Example (Use this while teaching)

```
Aluminum prices increased in Q3 due to supply shortages.
Energy costs also rose significantly impacting production.
Government policies further restricted exports in some regions.
Demand from the automotive sector remained strong.
```

👉 Tell students:
"We will apply all chunking strategies on this same example"

◇ 3. Types of Chunking (with Examples)

🔘 1. Fixed-Size Chunking

📌 Concept:

Split text into equal size

☑ Example:

```
Chunk 1:
Aluminum prices increased in Q3 due to supply shortages.
Energy costs also rose significantly

Chunk 2:
impacting production.
Government policies further restricted exports
```

👍 Advantage:

- Simple

👎 Disadvantage:

- Breaks sentence → loses meaning

🧠 Teaching Line:

👉 “Machine-friendly but not meaning-friendly”

🟣 2. Overlapping Chunking (VERY IMPORTANT 🔥)

📌 Concept:

Next chunk overlaps previous

☑️ Example:

```
Chunk 1:
Aluminum prices increased in Q3 due to supply shortages.
Energy costs also rose significantly

Chunk 2:
Energy costs also rose significantly impacting production.
Government policies further restricted exports
```

👍 Advantage:

- Preserves context
- Better retrieval

🧠 Teaching Line:

👉 “We repeat some part so that context is not lost”

🟣 3. Sentence-Based Chunking

📌 Concept:

Each sentence = chunk

☑️ Example:

```
Chunk 1:
Aluminum prices increased in Q3 due to supply shortages.

Chunk 2:
Energy costs also rose significantly impacting production.
```

👍 Advantage:

- Clean and meaningful

👎 Disadvantage:

- Uneven size

🧠 Teaching Line:

👉 “Human-readable chunks”

🟣 4. Paragraph-Based Chunking

📌 Concept:

Each paragraph = one chunk

☑️ Example:

```
Chunk 1:
Aluminum prices increased in Q3 due to supply shortages.
Energy costs also rose significantly impacting production.

Chunk 2:
Government policies further restricted exports in some regions.
Demand from the automotive sector remained strong.
```

👍 Advantage:

- Maintains structure

👎 Disadvantage:

- Too large sometimes

🟢 5. Semantic Chunking (Advanced 🔥)

📌 Concept:

Split based on meaning

☑️ Example:

```
Chunk 1 (Supply Issue):
Aluminum prices increased in Q3 due to supply shortages.

Chunk 2 (Cost Impact):
Energy costs also rose significantly impacting production.

Chunk 3 (Policy):
Government policies further restricted exports.

Chunk 4 (Demand):
Demand from the automotive sector remained strong.
```

👍 Advantage:

- Best retrieval quality

👎 Disadvantage:

- Complex

🧠 Teaching Line:

👉 “Instead of size, we split based on meaning”

🟣 6. Recursive Chunking (Production Level 🚀)

📌 Concept:

Try multiple splitting rules:

```
Paragraph → Sentence → Words
```

☑️ Example:

```
Chunk 1:
Aluminum prices increased in Q3 due to supply shortages.
Energy costs also rose significantly impacting production.

Chunk 2:
Government policies further restricted exports in some regions.
Demand from the automotive sector remained strong.
```

👍 Advantage:

- Balanced approach
- Used in LangChain

🧠 Teaching Line:

👉 “Smart chunking — adjusts automatically”

🟣 7. Token-Based Chunking

📌 Concept:

Split by token count

☑️ Example:

```
Chunk 1:
Aluminum prices increased in Q3

Chunk 2:
due to supply shortages energy costs
```

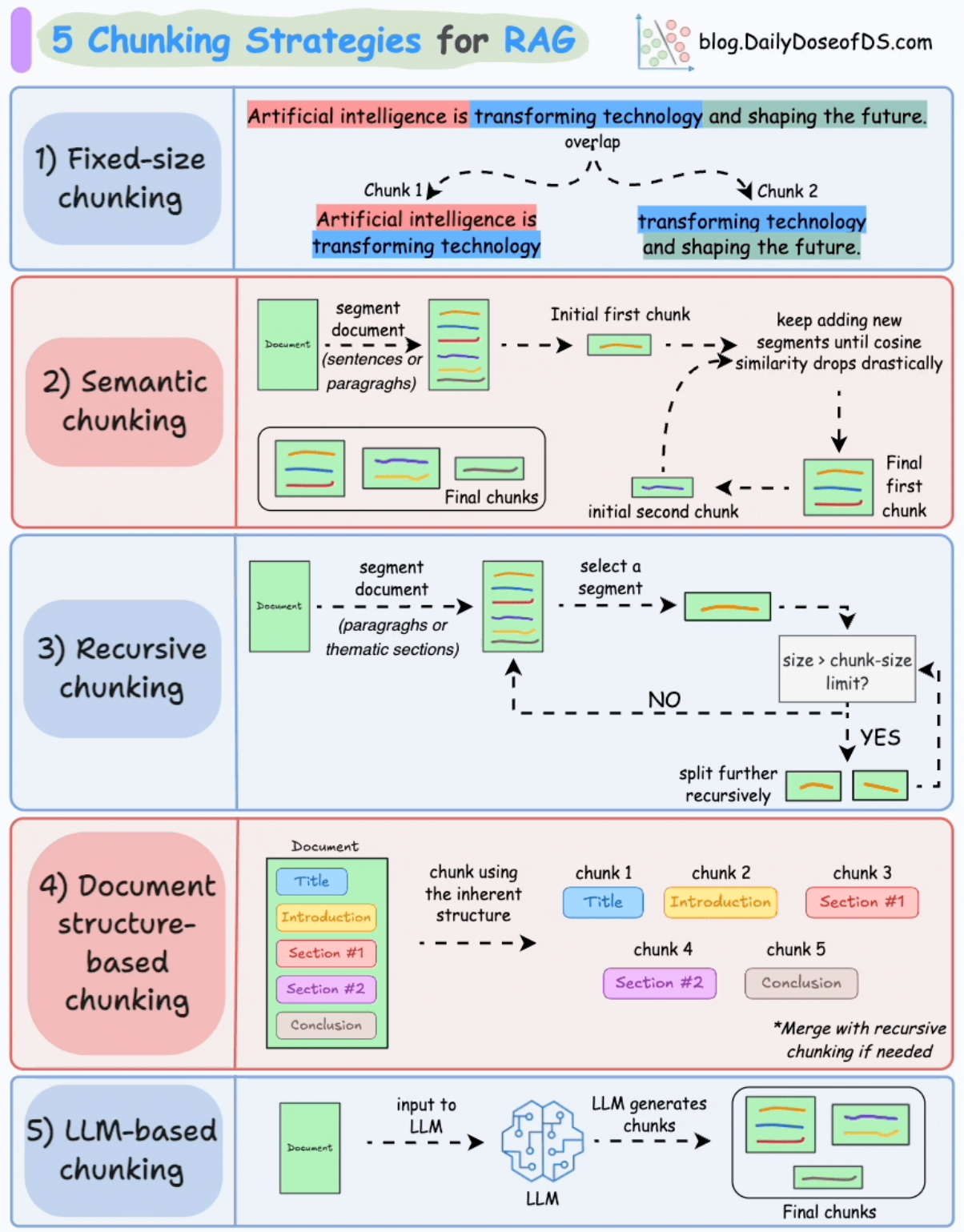
👎 Problem:

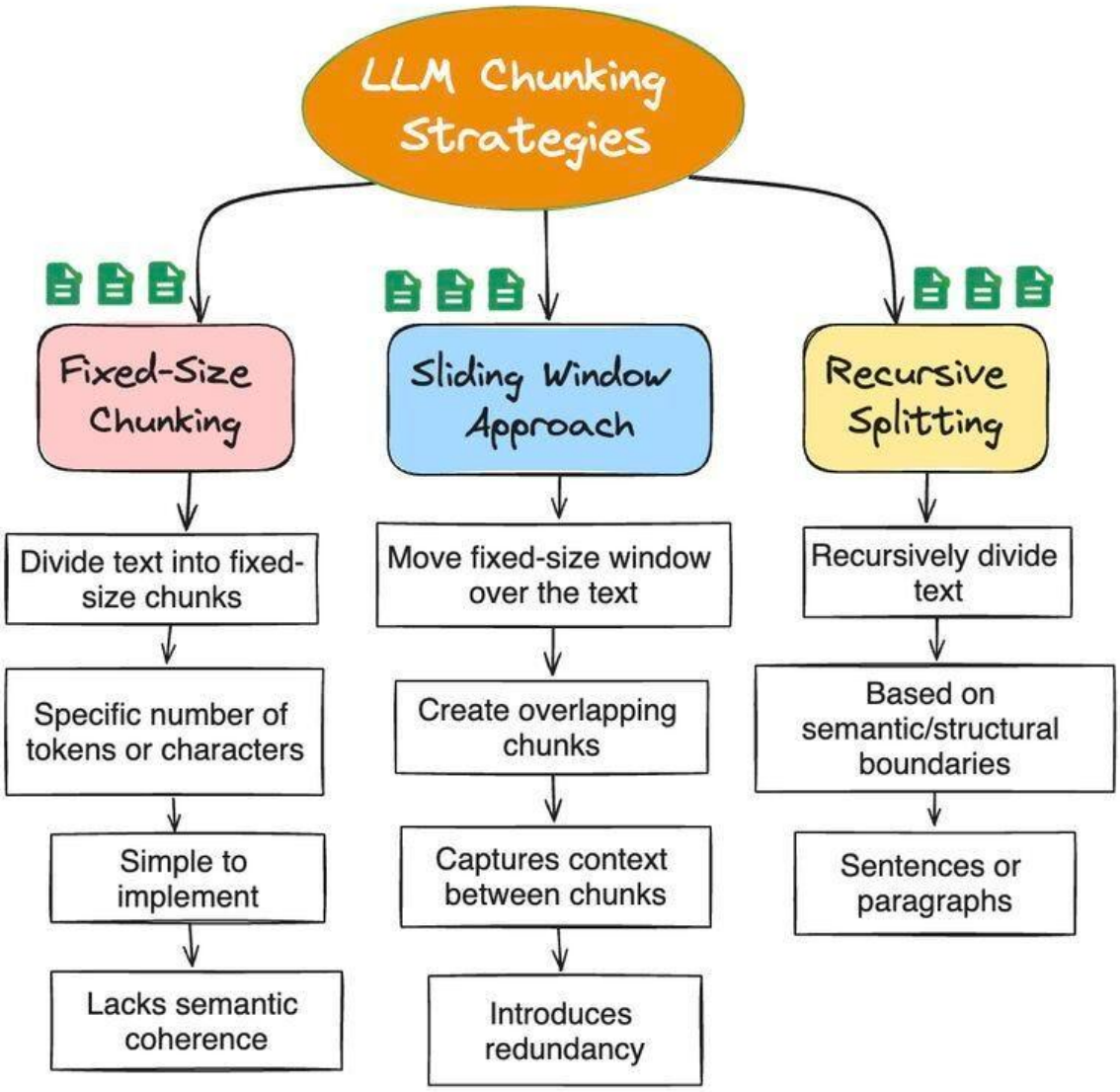
- Breaks meaning

🧠 Teaching Line:

👉 “LLM-friendly but not meaning-friendly”

🔷 4. Visual Understanding


5 Chunking Strategies for RAG
blog.DailyDoseofDS.com
1) Fixed-size chunking
Artificial intelligence is transforming technology and shaping the future.
overlap
Chunk 1
Artificial intelligence is transforming technology
Chunk 2
transforming technology and shaping the future.
2) Semantic chunking
Document
segment document
(sentences or paragraghs)
Initial first chunk
keep adding new segments until cosine similarity drops drastically
Final chunks
initial second chunk
Final first chunk
3) Recursive chunking
Document
segment document
(paragraghs or thematic sections)
select a segment
size > chunk-size limit?
NO
YES
split further recursively
4) Document structure-based chunking
Document
Title
Introduction
Section #1
Section #2
Conclusion
chunk using the inherent structure
chunk 1
Title
chunk 2
Introduction
chunk 3
Section #1
chunk 4
Section #2
chunk 5
Conclusion
*Merge with recursive chunking if needed
5) LLM-based chunking
Document
input to LLM
LLM
LLM generates chunks
Final chunks



4

◇ 5. Best Practice (Tell Students Important Point 🔥)

👉 In real-world systems:

- Chunk size → **300–800 tokens**
- Overlap → **10–20%**
- Strategy → **Recursive + Overlap**

◇ 6. Final Summary (Very Important for Notes)

```
Chunking improves retrieval quality in RAG.

Different strategies:
- Fixed → simple but breaks context
- Overlap → preserves context (most important)
- Sentence → clean but uneven
- Paragraph → structured but large
- Semantic → best but complex
- Recursive → production level
- Token → LLM constraint handling

Best approach = Recursive + Overlap
```

3. What is Re-ranking in RAG and why is it needed?

◇ 1. What is Re-Ranking?

📌 Definition:

Re-ranking is the process of re-ordering the retrieved chunks based on deeper relevance before sending them to the LLM.

🧠 Simple Idea:

- Retriever gives **Top-K results (approximate)**
- Re-ranker refines them → **more accurate order**

🎯 One-Line:

👉 "Retriever finds candidates, Re-ranker finds the best ones."

◇ 2. Why Do We Need Re-Ranking?

❌ Problem Without Re-Ranking

Retriever (vector search) uses **embedding similarity**, which is not perfect.

👉 It may return:

- Partially relevant chunks
- Irrelevant chunks
- Wrong ranking order

📊 Example

Query:

```
Why did aluminum prices increase?
```

Retrieved Chunks (Top 3):

```
Chunk 1:
Aluminum is widely used in construction.

Chunk 2:
Energy costs increased production expenses.

Chunk 3:
Supply shortages caused price increase.
```

❗ Problem:

- Chunk 1 is irrelevant but ranked first ❌
- Most relevant chunk is last ❌

◇ 3. What Re-Ranking Does

👉 Re-evaluates each chunk **with the query**

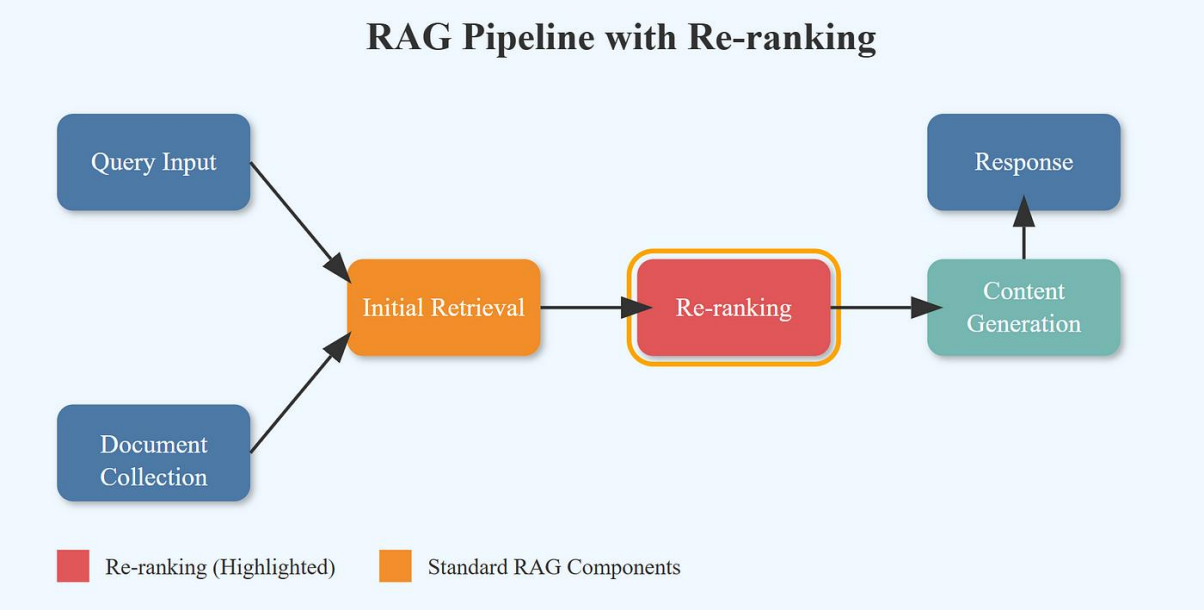
☑ After Re-Ranking:

```
Rank 1:
Supply shortages caused price increase.

Rank 2:
Energy costs increased production expenses.

Rank 3:
Aluminum is widely used in construction.
```

🎯 Result:

- Most relevant info goes to LLM
- Better answer quality

◇ 4. Visual Flow



◇ 5. How Re-Ranking Works (Technical)

🔘 Step 1: Retriever (Fast but Approximate)

- Uses vector DB (e.g., FAISS)
- Returns Top-K (e.g., 10 chunks)

🔘 Step 2: Re-Ranker (Slow but Accurate)

👉 Uses models like:

- Cross-encoder
- LLM-based scoring

🔍 What happens:

Instead of:

```
query → embedding → similarity
```

Re-ranker does:

```
(query + chunk) → relevance score
```

Example:

```
Input:
(Query, Chunk 1) → Score: 0.2
(Query, Chunk 2) → Score: 0.7
(Query, Chunk 3) → Score: 0.95
```

👉 Then sort by score

◇ 6. Types of Re-Ranking

🔘 1. Cross-Encoder Re-Ranking (MOST COMMON 🔥)

📌 Idea:

- Takes query + chunk together
- Understands deep relationship

Example:

```
"Why aluminum price increased?" + "Supply
shortage..." → High score
```

👍 Pros:

- Very accurate

👎 Cons:

- Slow

🔘 2. LLM-Based Re-Ranking

📌 Idea:

Use LLM to rank chunks

Example Prompt:

```
Rank the following chunks based on relevance to the
query.
```

👍 Pros:

- Very powerful

👎 Cons:

- Expensive

🥏 3. Hybrid Re-Ranking

📌 Idea:

- Combine keyword + semantic search

◇ 7. Where Re-Ranking Fits in RAG

```
User Query
    ↓
Embedding
    ↓
Retriever (Top-K chunks)
    ↓
🔥 Re-Ranker (Re-order Top-K)
    ↓
Top-N chunks
    ↓
LLM
    ↓
Final Answer
```

◇ 8. Real-World Example (Your Domain)

👉 Commodity Insights Use Case:

Query:

```
Why coal prices increased?
```

Without Re-Ranking:

- General industry info may come first ❌

With Re-Ranking:

- Supply shortage
- Logistics issue
- Policy changes

👉 LLM gets **high-quality context → better answer**

◇ 9. Why It is Critical (Interview Gold 🔥)

👉 Say this:

"Vector search is approximate and may not rank results perfectly. Re-ranking adds a second stage of deep semantic understanding, ensuring that the most relevant chunks are passed to the LLM, significantly improving answer accuracy."

◇ 10. When to Use Re-Ranking

Use when:

- Large documents

- High accuracy needed
- Enterprise RAG systems

Skip when:

- Small dataset
- Low latency requirement

🔥 Final Summary (For Notes)

```
Re-ranking improves retrieval quality in RAG.

Flow:
Retriever → Top-K → Re-ranker → Top-N → LLM

Why needed:
- Fix wrong ranking
- Remove irrelevant chunks
- Improve answer quality

Types:
- Cross-encoder (best)
- LLM-based
- Hybrid

Key idea:
Retriever is fast, re-ranker is accurate.
```

4. What are the common failure points in a RAG pipeline?

🧠 Common Failure Points in RAG Pipeline

◇ 1. Quick Overview (Where Failures Happen)

```
Data → Chunking → Embedding → Vector DB → Retrieval →
Re-ranking → LLM → Output
```

❌ ❌ ❌ ❌ ❌ ❌

👉 Failure can happen at **any stage**, not just the LLM.

💧 Key Insight (Very Important)

"Most RAG failures are due to bad retrieval, not bad generation."

◇ 2. Failure Points with Examples

🥏 1. Poor Data Quality

❌ Problem:

- Noisy, outdated, incomplete data

📌 Example:

```
Document:
"Aluminum prices may increase..." (old data)
```

User asks:
👉 "Current aluminum price trend?"

👉 Output = ❌ outdated answer

☑ Fix:

- Clean data
- Regular updates

🧠 Teaching Line:

👉 “Garbage in = Garbage out”

🟤 2. Bad Chunking Strategy

❌ Problem:

- Too large → irrelevant retrieval
- Too small → context lost

📌 Example:

```
Chunk:
"Aluminum prices increased... automotive demand
strong... policy changes..."
```

👉 Mixed topics → ❌ confusion

☑ Fix:

- Use **recursive + overlap**

🟤 3. Weak Embeddings

❌ Problem:

- Embeddings don’t capture meaning properly

📌 Example:

Query:

```
"Why price increased?"
```

Chunk:

```
"Supply shortage caused increase"
```

👉 If embedding weak → not matched ❌

☑️ Fix:

- Use better models
- Domain-specific embeddings

🔴 4. Retrieval Failure (MOST COMMON 🔥)

❌ Problem:

- Wrong chunks retrieved

📌 Example:

Query:

```
"Why aluminum prices increased?"
```

Retrieved:

```
"Aluminum is used in construction"
```

👉 Irrelevant ❌

☑ **Fix:**

- Improve chunking
- Better embeddings
- Hybrid search

🧠 **Teaching Line:**

👉 “If retrieval fails, everything fails”

🥈 **5. No Re-Ranking**

❌ **Problem:**

- Relevant chunk exists but ranked low

📌 **Example:**

```
Top 1 → General info
Top 3 → Actual answer
```

👉 LLM misses correct info ❌

☑️ **Fix:**

- Add re-ranking layer

🔘 **6. Context Overload (Token Limit Issue)**

❌ **Problem:**

- Too many chunks passed to LLM

📌 **Example:**

```
10 chunks → LLM confused
```

☑️ **Fix:**

- Limit Top-K
- Use summarization

🔘 **7. Prompt Design Issues**

❌ **Problem:**

- Poor prompt → LLM ignores context

📌 Example:

```
Bad Prompt:
Answer the question.
```

👉 LLM may hallucinate ❌

☑️ Fix:

```
Use only the given context. If not found, say "I
don't know."
```

🔘 8. Hallucination (LLM Failure)

❌ Problem:

- LLM generates info not in context

📌 Example:

Context:

```
No mention of policy
```

LLM:
👉 "Government policy caused increase" ❌

☑️ Fix:

- Strict prompting

- Grounded generation

🔘 9. Lack of Metadata Filtering

❌ Problem:

- Irrelevant time/source retrieved

📌 Example:

Query:

`"2025 aluminum price"`

Retrieved:

`2018 report` ❌

☑ Fix:

- Use metadata (date, source)

🔘 10. Latency Issues (Performance Failure)

❌ Problem:

- Too slow system

Causes:

- Large vector DB
- Re-ranking cost

☑ **Fix:**

- Caching
- Efficient indexing

5. How do you evaluate a RAG system? What metrics are used?

```
1. Retrieval Evaluation  → Are we fetching correct chunks?
2. Generation Evaluation → Is the answer correct?
3. End-to-End Evaluation → Does user get correct response?
```

🔥 **Golden Rule (Interview Line)**

"If retrieval is bad, generation will always be bad."

◇ **3. Retrieval Evaluation Metrics**

🟤 **1. Recall@K (MOST IMPORTANT 🔥)**

📌 **Definition:**

Out of all relevant documents, how many are retrieved in Top-K?

☑ Example:

Query:

```
Why aluminum price increased?
```

Relevant chunks:

```
[Supply shortage, Energy cost]
```

Retrieved Top-3:

```
[Supply shortage, Random text, Energy cost]
```

📊 Calculation:

```
Recall@3 = 2 / 2 = 1.0 ☑
```

🎯 Meaning:

- All relevant chunks found → Good retrieval

🥏 2. Precision@K

📌 Definition:

Out of retrieved chunks, how many are actually relevant?

Example:

Retrieved:

```
[Supply shortage, Random text, Energy cost]
```

Relevant:

```
2 out of 3
```

📊 Calculation:

```
Precision@3 = 2 / 3 = 0.66
```

🎯 Meaning:

- Higher precision → Less noise

🥈 3. MRR (Mean Reciprocal Rank)

📌 Definition:

Measures how early the correct answer appears

Example:

```
Rank 1 → Wrong
Rank 2 → Correct
```

📊 Calculation:

```
MRR = 1 / 2 = 0.5
```

🎯 Meaning:

- Higher = better ranking

🔘 4. Hit Rate

📌 Definition:

Did we retrieve at least one correct chunk?

Example:

```
Top-3 → contains correct chunk → Hit = 1
```

◇ 4. Generation Evaluation Metrics

🔘 1. Faithfulness (VERY IMPORTANT 🔥)

📌 Definition:

Is the answer grounded in retrieved context?

❌ Example:

Context:

```
No mention of policy
```

Answer:

```
Policy caused increase ❌
```

🎯 Meaning:

- Detect hallucination

🔘 2. Answer Relevance

📌 Definition:

Does the answer match the question?

Example:

Question:

`Why price increased?`

Answer:

`Aluminum is widely used` ❌

🔘 3. Context Relevance

📌 Definition:

Are retrieved chunks relevant to query?

🔘 4. Correctness (Ground Truth Based)

📌 Definition:

Compare with actual correct answer

◇ 5. End-to-End Evaluation

🟣 1. Exact Match (EM)

- Answer exactly matches ground truth

🟣 2. F1 Score

- Measures overlap between:
  - predicted answer
  - true answer

◇ 6. LLM-Based Evaluation (Modern Approach 🔥)

👉 Use LLM (like GPT-4) as judge

Example Prompt:

```
Evaluate if the answer is correct and grounded in
context.
Score from 1 to 5.
```

Used in:

- RAGAS
- TruLens

Tools:

- RAGAS
- TruLens

6. What is the difference between RAG and Fine-tuning — when to use which?

🧠 RAG vs Fine-Tuning (Teaching Notes)

◇ 1. Basic Definitions

🟣 What is RAG (Retrieval-Augmented Generation)?

RAG retrieves external data and gives it to the LLM at runtime to generate answers.

👉 Key idea:

- Knowledge is **outside the model**
- Fetched when needed

🟣 What is Fine-Tuning?

Fine-tuning updates the model's internal weights using new training data.

👉 Key idea:

- Knowledge is **inside the model**

◇ 2. Core Difference (Very Important)

```
RAG          → External Knowledge (Dynamic)
Fine-tuning → Internal Knowledge (Static)
```

Intuition (Best Way to Explain)

RAG = Open Book Exam

- You can look at documents while answering

Fine-tuning = Closed Book Exam

- You must remember everything

---

3. Detailed Comparison Table

Feature	RAG	Fine-Tuning
Knowledge Source	External (DB, docs)	Internal (model weights)
Data Update	Easy (just update DB)	Hard (retrain model)
Cost	Lower	High
Latency	Higher (retrieval step)	Lower
Accuracy	High (if retrieval good)	High (if trained well)
Hallucination	Reduced	Possible
Real-time Data	☑ Yes	✗ No
Use Case	QA, search, chatbots	Style, behavior, domain adaptation

---

4. Example (VERY IMPORTANT)

🎯 Use Case: Commodity Insights (Your Domain)

🟣 Using RAG

User asks:

```
Why aluminum prices increased in 2025?
```

👉 System:

- Retrieves latest reports
- Sends to LLM
- Generates answer

🟣 Using Fine-Tuning

Model trained on:

```
Historical commodity data patterns
```

👉 Output:

- Learns patterns like:
  - price trends
  - domain language

◇ 5. When to Use RAG?

☑ Use RAG when:

- Data changes frequently
- Need real-time information

- Working with:
  - PDFs
  - Databases
  - Internal docs

📌 Examples:

- Chat with documents
- Knowledge base QA
- Customer support bots

◇ 6. When to Use Fine-Tuning?

☑ Use Fine-Tuning when:

- You want to change:
  - Writing style
  - Tone
  - Behavior

📌 Examples:

- Legal assistant tone
- Medical domain language
- Code generation style

◇ 7. When to Use BOTH (Advanced 🔥)

👉 Best systems use **RAG + Fine-Tuning together**

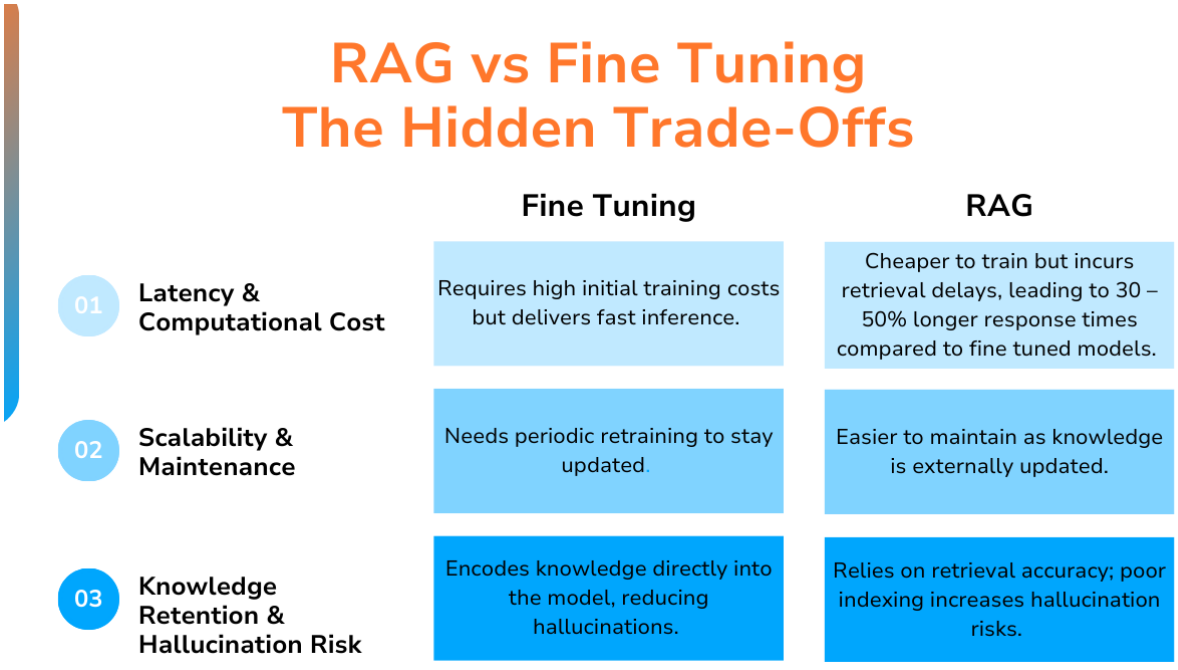
Example:

- Fine-tune model for:
  - Domain understanding
- Use RAG for:
  - Latest data

🎯 Result:

- Smart + up-to-date system

◇ 8. Visual Understanding



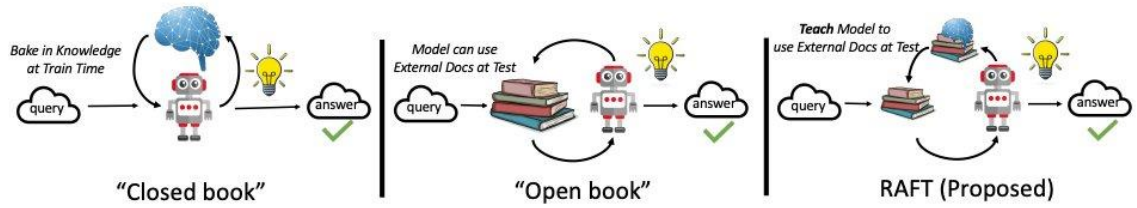
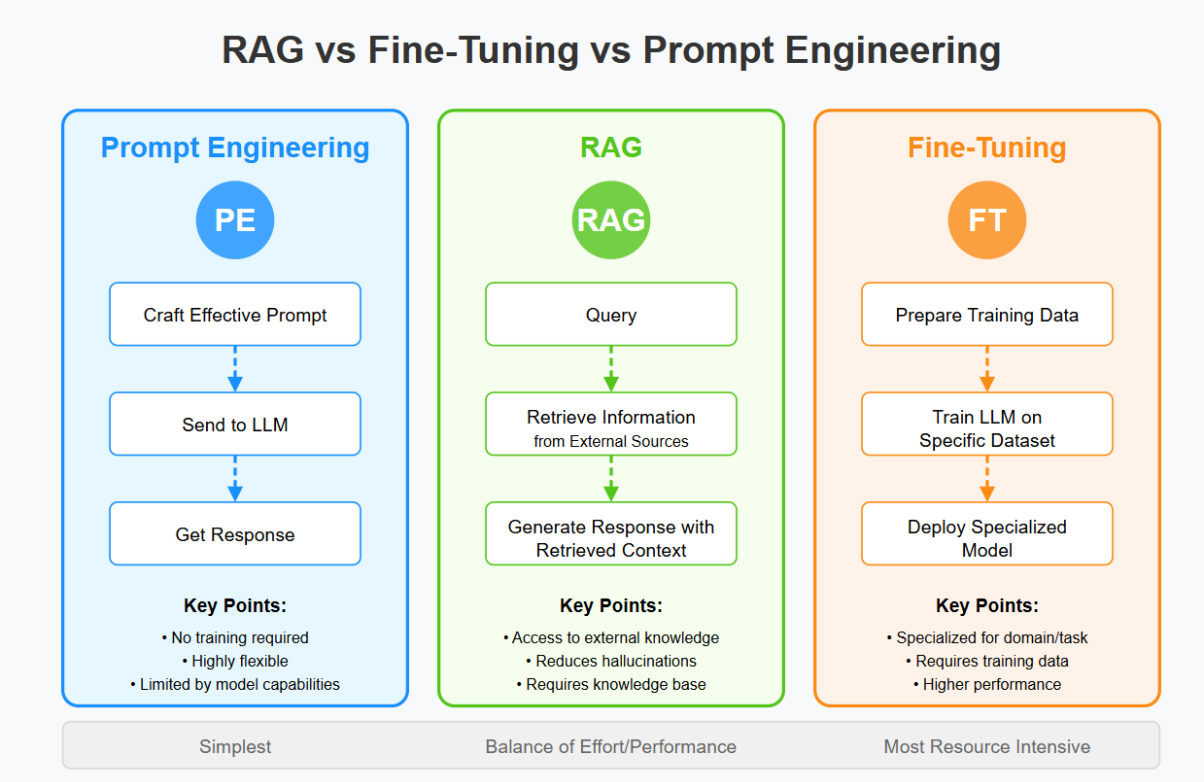


Figure 1: **How best to prepare for an Exam?**(a) Fine-tuning based approaches implement "studying" by either directly "memorizing" the input documents or answering practice QA without referencing the documents. (b) Alternatively, in-context retrieval methods fail to leverage the learning opportunity afforded by the fixed domain and are equivalent to taking an open-book exam without studying. While these approaches leverage in-domain learning, they fail to prepare for open-book tests. In contrast, our approach (c) RAFT leverages fine-tuning with question-answer pairs while referencing the documents in a simulated imperfect retrieval setting — thereby effectively preparing for the open-book exam setting.



◇ 9. Common Mistake (Very Important)

❌ People think:

"Fine-tuning can replace RAG"

👉 Reality:

- Fine-tuning ≠ real-time knowledge
- RAG ≠ behavior learning

Interview Killer Answer

If asked:

👉 *"Difference between RAG and Fine-tuning?"*

Say:

"RAG retrieves external knowledge at runtime and provides dynamic, up-to-date answers, while fine-tuning updates the model's internal weights to learn domain-specific behavior or style. RAG is preferred when data changes frequently, whereas fine-tuning is used when we want to customize model behavior. In practice, both are often combined for best results."